

B.ComAllied2023-24

FIRSTYEAR–SEMESTER–I

PROGRAMMING IN C AND LAB									
Subject Code	L	T	P	S	Credits	Inst. Hours	Marks		
							CIA	External	Total
23BCOA 1		2	2		3	4	25	75	100
Learning Objectives									
LO1	Describe the core syntax and semantics of C programming language.								
LO2	Discover the need for working with the strings and functions.								
LO3	Illustrate the process of structuring the data using matrix, struct.								
Prerequisites: Should have studied Commerce in XII Std									
	Contents								No. of Hours
Unit I	Introduction to C Language: C Language Introduction-Features of C Language-Benefits of C over other languages-Compilation of C Program-First Programming C Pre-processor in C Pre-Processor directives								
Unit II	Variables, Data Types & Operators: Variables and Keywords in C-Scope rules in C-Data Types in C-Operators & Its Types- Type casting in C								
Unit III	Control Flow Statements: Decision Making Statements-Switch Statement in C-C Loops & Control Structure Practice problems-Continue Statement , Break Statement Array & String Handling in C:ArraysinC-StringsinC								
Unit IV	Multidimensional Arrays in C-String functions in C- Practice problems Functions in C:Function Prototype-Parameter Passing Techniques in C-Storage Classes in C-Recursion Concept - Functions in C Practice problems								
Unit V	Pointers, Structures, and Unions: Pointers in C-Structures-Union -Enumeration(or enum) in C-Pointer vs Array in C–C application programs(Sorting, Matrix manipulations, student’s Mark list preparation)								
	Total								
Course Outcomes									
CO1	Apply the concept of Control Structures to solve any given problem.								
CO2	Apply the concept of single and multi-dimensional arrays to solve problems Related to searching, sorting and matrix operations.								
CO3	ApplytheconceptofStringsforwritingprogramsrelatedtocharacterarray.								
CO4	Write programs using concept of user defined and recursive functions.								
CO5	Apply concept of structures to write programs.								

Unit I

Introduction to C Language:

C Language Introduction

- C is a **high-level programming language** developed by **Dennis Ritchie** at **AT&T Bell Laboratories (USA)** in 1972.
- It was developed for writing **system software**, especially the **UNIX operating system**.
- C is known as the “**Mother of all languages**” because many modern languages like C++, Java, and Python are based on it.

C combines the features of both:

- **Low-level language** (close to hardware)
- **High-level language** (easy to understand)

Features of C Language

The **C language** is one of the most popular and powerful programming languages. It provides a variety of features that make it suitable for both **system** and **application software development**.

1. Simple and Easy to Learn

- C language has a small set of keywords and clear syntax, making it easy for beginners to understand and use.

2. Structured Programming Language

- C supports **structured programming**, which means a program can be divided into smaller functions or modules.
- This makes the program **easy to debug, test, and maintain**.

3. Portable

- C programs are **machine-independent**.
- Once written, the same program can run on any computer with minimal or no changes.

4. Fast and Efficient

- C is a **compiled language**, so the code is translated directly into machine code, which makes execution **very fast** compared to interpreted languages like Python.

5. Rich Library

- C provides a large collection of **built-in functions** for performing various tasks (e.g., input/output, string handling, math operations).

6. Low-level Access

- C allows direct access to **memory** using **pointers**, enabling programmers to write **system-level programs** like operating systems and compilers.

7. Extensible

- Users can define their own functions, making it easy to **extend the capabilities** of the language.

8. Recursion

- C supports **recursive functions**, where a function can call itself to solve repetitive problems.

9. Middle-Level Language

- C combines features of both **low-level (assembly)** and **high-level (user-friendly)** languages — that's why it's called a **middle-level language**.

10. Dynamic Memory Management

- Using `malloc()`, `calloc()`, and `free()`, C allows programmers to **allocate and release memory** at runtime.

Feature	Description
Simple	Easy to write and understand
Structured	Supports modular programming
Portable	Works across different systems
Fast	Compiled, efficient execution
Rich Library	Many built-in functions
Low-level Access	Uses pointers
Extensible	User-defined functions
Recursive	Supports recursion
Middle-level	Combines high & low-level features
Dynamic Memory	Runtime memory allocation

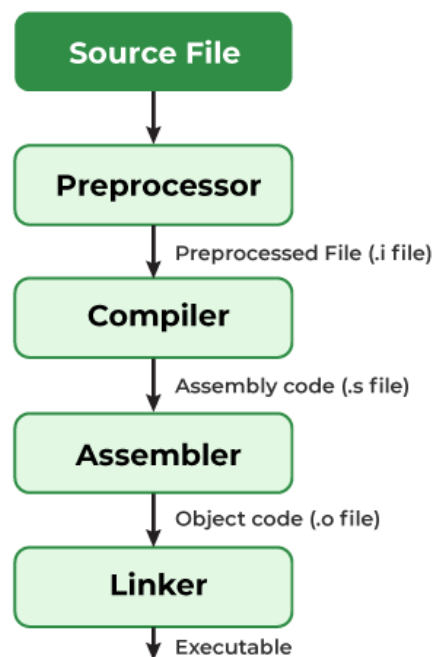
Benefits of C over other languages

Benefit	Explanation
Speed	C executes faster than many interpreted languages like Python.
Portability	Same code can be compiled on multiple platforms.

Modularity	Programs can be broken into reusable modules (functions).
Powerful	Can access and manipulate hardware directly.
Foundation for Other Languages	Languages like C++, Java, and C# are based on C.
Flexibility	Suitable for both system software and application software.

Compilation of C Program

- The compilation is the process of converting the source code of the C language into machine code. As C is a mid-level language, it needs a compiler to convert it into an executable code so that the program can be run on our machine.
- The C program goes through the following phases during compilation:



Compile and run a C program?

We first need a compiler and a code editor to compile and run a C Program. The below example is of an Ubuntu machine with GCC compiler.

Step 1: Creating a C Source File

We first create a C program using an editor and save the file as filename.c In linux, we can use vi to create a file from the terminal using the command:

```
vi filename.c
```

In windows, we can use the Notepad to do the same. Then write a simple hello world program and save it.

Step 2: Compiling using GCC compiler

We use the following command in the terminal for compiling our **filename.c** source file.

```
gcc filename.c -o filename
```

We can pass many instructions to the GCC compiler to different tasks such as:

- The option `-Wall` enables all compiler's warning messages. This option is recommended to generate better code.
- The option `-o` is used to specify the output file name. If we do not use this option, then an output file with the name `a.out` is generated.

If there are no errors in our C program, the executable file of the C program will be generated.

Step 3: Executing the program

After compilation executable is generated and we run the generated executable using the below command.

```
./filename // for linux  
filename // for windows
```

The program will be executed, and the output will be shown in the terminal.

Phases of Compilation:

1. **Editing:**
 - Writing the source code using an editor like Notepad or Code::Blocks.
 - File saved with `.c` extension (example: `program.c`).
2. **Preprocessing:**
 - Handles **preprocessor directives** (like `#include`, `#define`).
 - Expands macros and includes necessary header files.
3. **Compilation:**
 - Converts source code into **assembly language**.
 - Checks for syntax errors.
4. **Assembly and Linking:**
 - Converts code into **object file (.obj / .o)** and links it with library files.
 - The **linker** produces the final **executable file (.exe)**.
5. **Execution:**
 - The loader loads the program into memory and executes it.
 - Output is displayed on the screen.

First Programming C Pre-processor in C Pre-Processor directives

Example: Hello World Program

```
#include <stdio.h> // Standard Input/output header file
```

```
int main() {      // Main function where execution starts
    printf("Hello, World!"); // Prints output
    return 0;      // Returns control to operating system
}
```

Output:

Hello, World!

Explanation:

- `#include <stdio.h>` → Adds standard library for input/output.
- `main()` → Every C program must have a main function.
- `printf()` → Displays text or output on the screen.
- `return 0;` → Indicates successful completion of the program.

*****End*****

Unit 1 Questions

2 Marks Questions

1. Who developed the C language?
 2. What is a compiler?
 3. Define a variable.
 4. What is the use of #include directive?
 5. What is the function of return 0; in C?
-

5 Marks Questions

1. Explain the main features of C language.
 2. Describe the benefits of C over other languages.
 3. Explain the phases of compilation in C.
 4. Write and explain a simple C program to display "Hello, World!".
 5. Explain different preprocessor directives used in C.
-

10 Marks Questions

1. Explain in detail the structure of a C program with an example.
2. Describe the compilation and execution process of a C program.
3. Explain the features and advantages of C language in detail.
4. Discuss in detail about C preprocessor and its directives with examples.
5. Explain variables, constants, data types, and keywords in C with examples.

Unit II

Variables,

A **variable** in C is a **named memory location** used to store a value that can be changed during program execution.

Syntax

```
data_type variable_name = value;
```

Example:

```
int age = 20;
```

Here:

- `int` → data type
- `age` → variable name
- `20` → value stored in memory

Characteristics of Variables

1. Variable names are **identifiers**.
2. Each variable has a **data type** (defines the kind of value it stores).
3. The **value of a variable can change** while the program runs.
4. Every variable has a **memory address** where its data is stored.

Rules for Naming Variables

1. Must begin with a letter or underscore (`_`).
2. Can contain **letters, digits, and underscores**.
3. No special characters or spaces allowed.
4. Must **not be a keyword**.
5. Variable names are **case-sensitive** (`Total` ≠ `total`).
6. Should be **meaningful** (e.g., `marks`, `sum`, `age`).

Valid Names:

```
sum, _value, total Marks, number1
```

Invalid Names:

```
2total, first-name, float
```

Example:

```
int num;  
float rate = 8.5;  
char grade = 'A';
```

Program – Declaring and Using Variables

```
#include <stdio.h>
```

```

int main() {
    int age = 20;           // integer variable
    float height = 5.8;     // float variable
    char grade = 'A';       // character variable

    printf("Age: %d\n", age);
    printf("Height: %.1f\n", height);
    printf("Grade: %c\n", grade);

    return 0;
}

```

Output:

```

Age: 20
Height: 5.8
Grade: A

```

Data Types & Operators:

A **data type** defines the type of data that can be stored in a variable and the amount of memory required.

Integer Data Type

- Stores whole numbers (positive, negative, or zero).
- We use **int keyword** to declare the integer variable:
- Size: 4 bytes, Range: -2,147,483,648 to 2,147,483,647.
- Format specifier: %d.

Character Data Type

- Stores a single character (like 'A', 'b', or '5').
- Size: 1 byte, Range: -128 to 127 (signed by default).
- Format specifier: %c.

Float Data Type

- Stores **decimal numbers** (numbers with fractional part).
- Size: 4 bytes, Approximate range: 3.4e-38 to 3.4e+38.
- Format specifier: %f.

Double Data Type

- Stores decimal numbers with more precision than float.
- Size: 8 bytes, Approximate range: 1.7e-308 to 1.7e+308.
- Format specifier: %lf.

Void Data Type

- Represents no value or empty type.

- Used in functions that does not return any value.
- Can also be used for generic pointers (void *) in memory operations.

Category	Data Type	Description	Example	Size
Primary (Basic)	int, float, char, double	Basic data types	int a=10;	1–8 bytes
Derived	array, pointer, structure, union	Derived from basic types	int arr[5];	Depends
Enumeration	enum	User-defined constants	enum day{ Mon, Tue};	2–4 bytes
Void	void	No value / empty	void func();	—

Operators in C

An **operator** in C is a special symbol that tells the compiler to perform a specific mathematical or logical operation on one or more operands.

Example:

a + b

Here,

- + → Operator
- a, b → Operands

The result of the operation is a + b.

2. Types of Operators in C

C language provides several types of operators:

1. **Arithmetic Operators**
2. **Relational Operators**
3. **Logical Operators**
4. **Assignment Operators**
5. **Increment and Decrement Operators**
6. **Conditional (Ternary) Operator**
7. **Bitwise Operators**
8. **Special Operators**

1. Arithmetic Operators

Used to perform mathematical calculations.

Operator	Description	Example	Result
+	Addition	a + b	15
-	Subtraction	a - b	5
*	Multiplication	a * b	50
/	Division	a / b	2
%	Modulus (Remainder)	a % b	0

Example Program:

```
#include <stdio.h>
int main() {
    int a = 10, b = 5;
    printf("Addition = %d\n", a + b);
    printf("Subtraction = %d\n", a - b);
    printf("Multiplication = %d\n", a * b);
    printf("Division = %d\n", a / b);
    printf("Modulus = %d\n", a % b);
    return 0;
}
```

2. Relational Operators

Used to compare two values.

The result is **1 (true)** or **0 (false)**.

Operator	Description	Example	Result
==	Equal to	a == b	0
!=	Not equal to	a != b	1
>	Greater than	a > b	1
<	Less than	a < b	0
>=	Greater than or equal to	a >= b	1
<=	Less than or equal to	a <= b	0

Example:

```
if (a > b)
    printf("a is greater");
else
    printf("b is greater");
```

3. Logical Operators

Used to combine two or more conditions.

Operator	Description	Example	Result
&&	Logical AND	(a > b && b < c)	True if both true
^		^	Logical OR
!	Logical NOT	! (a > b)	True if false

Example Program:

```
#include <stdio.h>

int main() {
    int a = 10, b = 5;

    printf("%d\n", (a > b) && (b < 10)); // 1
    printf("%d\n", (a < b) || (b == 5)); // 1
    printf("%d\n", !(a == 10));          // 0
    return 0;
}
```

4. Assignment Operators

Used to assign values to variables.

Operator	Example	Equivalent To
=	a = 10	Assigns 10 to a
+=	a += 5	a = a + 5
-=	a -= 5	a = a - 5
*=	a *= 5	a = a * 5
/=	a /= 5	a = a / 5
%=	a %= 5	a = a % 5

5. Increment and Decrement Operators

Used to increase or decrease a variable by 1.

Operator	Description	Example	Result
++	Increment	a++ or ++a	Increases a by 1
--	Decrement	a-- or --a	Decreases a by 1

Example:

```
int a = 5;
printf("%d\n", a++); // Prints 5, then a becomes 6
printf("%d\n", ++a); // a becomes 7, then prints 7
```

6. Conditional (Ternary) Operator

A shorthand for an if-else statement.

Syntax:

```
(condition) ? expression1 : expression2;
```

Example:

```
int a = 10, b = 20;
```

```
int max = (a > b) ? a : b;
printf("Maximum = %d", max);
```

Output:

Maximum = 20

7. Bitwise Operators

Used to perform bit-level operations.

Operator	Description	Example	Result (if a=5, b=3)
&	AND	a & b	1
	OR	a b	7
^	XOR	a ^ b	6
~	NOT	~a	-6
<<	Left Shift	a << 1	10
>>	Right Shift	a >> 1	2

8. Special Operators

Operator	Description	Example
sizeof	Returns size of a variable	sizeof(int)
&	Address of variable	&a
*	Pointer (value at address)	*ptr
->	Access structure member using pointer	ptr->name
.	Access structure member	struct.name

Example Program (Combined Operators)

```
#include <stdio.h>
int main() {
    int a = 10, b = 3, result;

    result = (a > b) ? (a + b) : (a - b);
    printf("Result = %d\n", result);

    result = a & b;
    printf("Bitwise AND = %d\n", result);

    printf("Size of int = %lu bytes\n", sizeof(a));
    return 0;
}
```

Output:

Result = 13
 Bitwise AND = 2
 Size of int = 4 bytes

Summary Table

Type	Example Operators	Purpose
Arithmetic	+, -, *, /, %	Mathematical calculations
Relational	>, <, ==, !=	Comparisons
Logical	&&, , !	Combine conditions
Assignment	=, +=, -=	Assign values
Increment/Decrement	++, --	Increase/decrease value
Conditional	?:	Decision making
Bitwise	&, , ^, ~, <<, >>	Bit-level operations
Special	sizeof, &, *, -, >, .	Miscellaneous

Variables

- In C language, variables are the storage place where some form of data is stored. Different variables require different amount of memory on which a set of operations is applied.
- A variable name cannot start with a number. It can consist of alphabets, number, underscore “_”.

Syntax of declaring variables in C language

type variable_name;

Keywords in C

- ✓ **Keywords** are predefined or reserved words that have special meanings to the compiler. These are part of the syntax and cannot be used as identifiers in the program.
- ✓ A list of keywords in C or reserved words in the C programming language are mentioned below:

auto	break	case	char	const	continue	default	do
double	else	enum	extern	float	for	goto	if
int	long	register	return	short	signed	sizeof	static
struct	switch	typedef	union	unsigned	void	volatile	while

Category	Keywords
Data Type Keywords	char, int, float, double, void, short, long, signed, unsigned
Operator & Utility Keywords	sizeof, return, goto, typedef
Control Flow Keywords	if, else, switch, case, default, for, while, do, break, continue
Storage Class Keywords	auto, register, static, extern
Type Qualifiers	const, volatile
User Defined Types	struct, union, enum

Scope rules in C

The scope of a variable in C is the block or the region in the program where a variable is declared, defined, and used. Outside this region, we cannot access the variable, and it is treated as an undeclared identifier.

- The scope of an identifier is the part of the program where the identifier may directly be accessible.
- All variables are lexically (or statically) scoped in C which means the scope is defined at the compiler time and not dependent on the caller of the function.

Types of Scope in C

There are **four types of variable scopes** in C:

1. **Local Scope**
2. **Global Scope**
3. **Block Scope**
4. **Function Scope**

1 Local Scope

- Variables declared **inside a function or block** are called **local variables**.
- They are **created when the function is called** and **destroyed when it ends**.
- Accessible **only within that function**.

Example:

```
#include <stdio.h>
void display() {
    int x = 10; // Local variable
    printf("x = %d\n", x);
}
int main() {
    display();
}
```

```

        // printf("%d", x); // ✗ Error: x is not visible here
    return 0;
}

```

Output:

x = 10

2 Global Scope

- Variables declared **outside all functions** are called **global variables**.
- They can be **used by all functions** in the program.
- Their value **remains until the program ends**.

Example:

```

#include <stdio.h>
int num = 100; // Global variable

void display() {
    printf("Inside function: %d\n", num);
}
int main() {
    printf("In main: %d\n", num);
    display();
    return 0;
}

```

Output:

In main: 100
Inside function: 100

3 Block Scope

- A variable declared **inside a block { }** is visible **only inside that block**.
- Also known as **compound statement scope**.

Example:

```

#include <stdio.h>
int main() {
    int a = 10;
    {
        int b = 20;
        printf("a = %d, b = %d\n", a, b);
    }
    // printf("%d", b); // ✗ Error: b not visible outside
    block
    return 0;
}

```

Output:

a = 10, b = 20

4 Function Scope

- The **name of a function** itself has **file scope** (global by default).

- It can be used anywhere in the same file **after its declaration**.

Example:

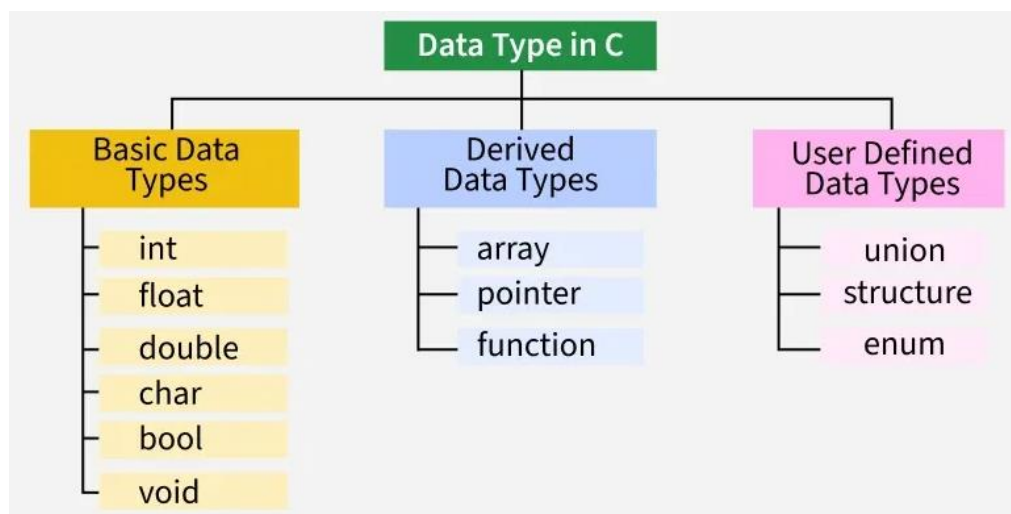
```
#include <stdio.h>
void greet() { // Function scope
    printf("Hello C Programmer!");
}
int main() {
    greet(); // Function accessible anywhere
    return 0;
}
```

Output:

Hello C Programmer!

Data Types in C

- Each variable in C has an associated data type. It specifies the type of data that the variable can store like integer, character, floating, double, etc.
- C is a statically type language where each variable's type must be specified at the declaration and once specified, it cannot be changed.



Integer Data Type

- Stores whole numbers (positive, negative, or zero).
- We use **int keyword** to declare the integer variable:
- Size: 4 bytes, Range: -2,147,483,648 to 2,147,483,647.
- Format specifier: %d.

```
#include <stdio.h>
```

```
int main() {  
    int var = 22;  
  
    printf("var = %d", var);  
    return 0;  
}
```

Character Data Type

- Stores a single character (like 'A', 'b', or '5').
- Size: 1 byte, Range: -128 to 127 (signed by default).
- Format specifier: %c.

```
#include <stdio.h>
```

```
int main() {  
    char ch = 'A';  
  
    printf("ch = %c", ch);  
    return 0;  
}
```

Float Data Type

- Stores **decimal numbers** (numbers with fractional part).
- Size: 4 bytes, Approximate range: 3.4e-38 to 3.4e+38.
- Format specifier: %f.

```
#include <stdio.h>
```

```
int main() {  
    float val = 12.45;  
  
    printf("val = %f", val);  
    return 0;  
}
```

Double Data Type

- Stores decimal numbers with more precision than float.
- Size: 8 bytes, Approximate range: 1.7e-308 to 1.7e+308.
- Format specifier: %lf.

```
#include <stdio.h>
```

```
int main() {
```

```
double val = 1.4521;
```

```
printf("val = %lf", val);
```

```
return 0;
```

```
}
```

Void Data Type

- Represents no value or empty type.
- Used in functions that does not return any value.
- Can also be used for generic pointers (void *) in memory operations.

```
#include <stdio.h>
```

```
// Function with void return type
```

```
void greet()
```

```
{
```

```
    printf("Hello, welcome!\n");
```

```
}
```

```
int main()
```

```
{
```

```
    greet();
```

```
    return 0;
```

```
}
```

Operators & Its Types- Type casting in C

Types of Operators in C

Category	Operators	Description
1. Arithmetic Operators	+ - * / %	Perform mathematical calculations
2. Relational Operators	== != > < >= <=	Compare two values
3. Logical Operators	`&&	
4. Assignment Operators	= += -= *= /= %=	Assign or modify values
5. Increment/Decrement Operators	++ --	Increase or decrease a value by 1
6. Bitwise Operators	`&	^ ~ << >>`
7. Conditional (Ternary) Operator	?:	Short form of if-else
8. Comma Operator	,	Separates multiple expressions
9. Special Operators	sizeof , &, *	Used for size, address, and pointer operations

1. Arithmetic Operators

Example:

```
#include <stdio.h>
int main() {
    int a = 10, b = 4;
    printf("Addition: %d\n", a + b);
    printf("Subtraction: %d\n", a - b);
    printf("Multiplication: %d\n", a * b);
    printf("Division: %d\n", a / b);
    printf("Modulus: %d\n", a % b);
    return 0;
}
```

Output:

```
Addition: 14
Subtraction: 6
Multiplication: 40
Division: 2
Modulus: 2
```

2. Relational Operators

Used to compare two values and return **true (1)** or **false (0)**.

Example:

```
#include <stdio.h>
int main() {
    int x = 5, y = 10;
    printf("%d\n", x > y);
    printf("%d\n", x < y);
    printf("%d\n", x == y);
    return 0;
}
```

Output:

```
0
1
0
```

3. Logical Operators

Used to combine multiple conditions.

Operator	Meaning
&&	Logical AND
!	Logical NOT

Example:

```
#include <stdio.h>
int main() {
    int a = 5, b = 10;
    printf("%d\n", (a < b) && (b > 0)); // True
    printf("%d\n", (a > b) || (b > 0)); // True
    printf("%d\n", !(a == b));          // True
    return 0;
}
```

Output:

1
1
1

4. Assignment Operators

Used to assign or modify values.

Operator	Example	Equivalent
=	a = 5	—
+=	a += 2	a = a + 2
-=	a -= 3	a = a - 3

Example:

```
#include <stdio.h>
int main() {
    int x = 10;
    x += 5;
    printf("x = %d", x);
    return 0;
}
```

Output:

x = 15

5. Increment and Decrement Operators

Operator	Meaning
++	Increases value by 1
--	Decreases value by 1

Example:

```
#include <stdio.h>
int main() {
    int a = 5;
```

```

printf("%d\n", ++a); // Pre-increment
printf("%d\n", a++); // Post-increment
printf("%d\n", a);   // Final value
return 0;
}

```

Output:

```

6
6
7

```

Conditional (Ternary) Operator

Syntax:

```
condition ? expression1 : expression2;
```

Example:

```

#include <stdio.h>

int main() {
    int num = 10;
    (num % 2 == 0) ? printf("Even") : printf("Odd");
    return 0;
}

```

Output:

```
Even
```

Type Casting in C

- Type casting is **converting one data type into another**.
- It helps ensure correct data handling during operations.

Types of Type Casting

Type	Description	Example
Implicit Casting (Type Conversion)	Done automatically by the compiler	int a = 5; float b = a;
Explicit Casting (Type Casting)	Done manually by the programmer	float b = (float)a;

Example 2: Explicit Type Casting

```

#include <stdio.h>

int main() {
    int a = 7, b = 2;

```

```
float result = (float)a / b;  
printf("Result = %.2f", result);  
return 0;  
}
```

Output:

Result = 3.50

*****End*****

UNIT II Most Important Questions

10 Two-Mark Questions

1. What is a variable in C?
2. Define keyword with example.
3. What is the scope of a variable?
4. Define data type.
5. Write the syntax of declaring a variable.
6. List any four types of operators in C.
7. What is type casting?
8. Differentiate between implicit and explicit type casting.
9. Mention any four keywords in C.
10. What is the difference between local and global variables?

5 Five-Mark Questions

1. Explain the **rules for naming and declaring variables** in C with examples.
2. Write short notes on **scope rules in C** with an example program.
3. Describe the **different types of data types** used in C language.
4. Explain **types of operators** in C with suitable examples.
5. What is **type casting**? Explain **implicit and explicit type casting** with examples.

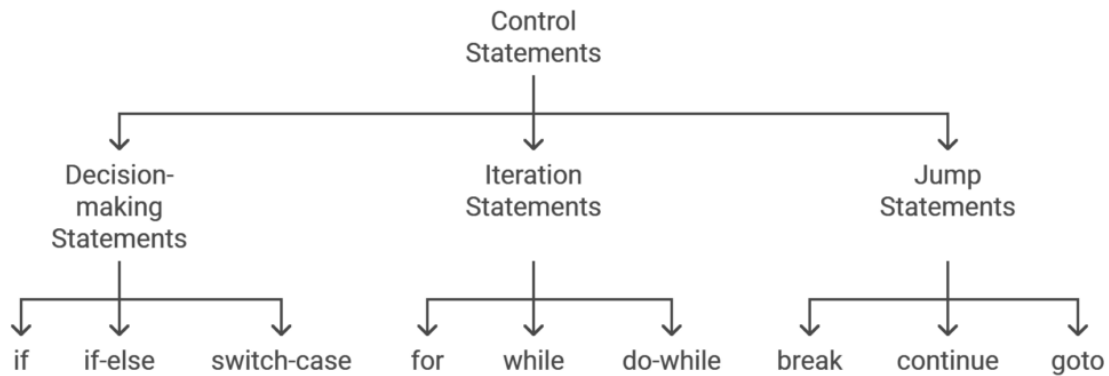
5 Ten-Mark Questions

1. Discuss in detail the **different data types** in C with syntax, memory size, and examples.
2. Explain **scope rules and lifetime of variables** in C with suitable code examples.
3. Describe **all types of operators** in C in detail with syntax and example programs.
4. Explain **type conversion and type casting** in detail with example programs.
5. Discuss **variables and keywords in C**, their declaration, initialization, and rules with examples.

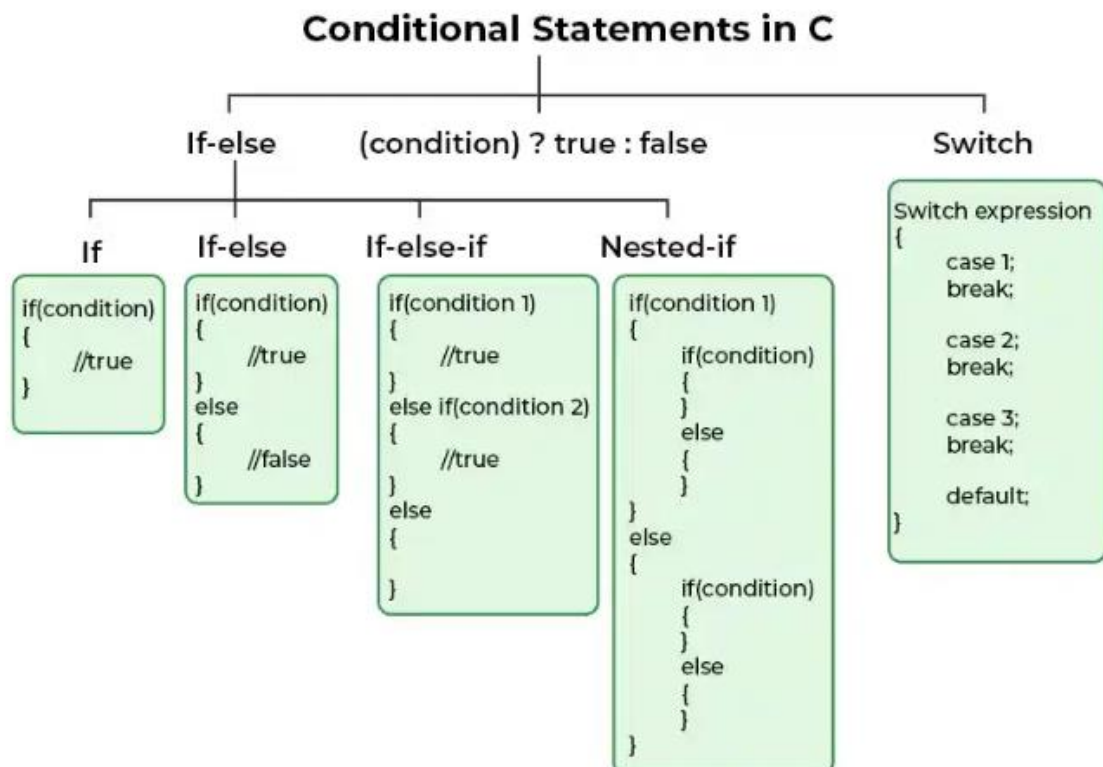
Unit III

Control Flow Statements:

- ✓ **Control flow** refers to the order in which statements within a program execute. While programs typically follow a **sequential flow** from top to bottom.



1. Decision Making Statements



(a) if Statement

- Used when we need to execute a block of code only if a condition is **true**.

```
if (marks >= 50) {  
    printf("Pass");  
}
```

(b) if-else Statement

- Provides **two-way decision**: one block if true, another if false.

```
if (marks >= 50) {  
    printf("Pass");  
} else {  
    printf("Fail");  
}
```

(c) Nested if Statement

- An if inside another if.

```
if (marks >= 50) {  
    if (marks >= 75) {  
        printf("Distinction");  
    }  
}
```

(d) else-if Ladder

- Used when there are **multiple conditions**.

```
if (marks >= 90)  
    printf("Grade A");  
else if (marks >= 75)  
    printf("Grade B");  
else if (marks >= 50)  
    printf("Grade C");  
else  
    printf("Fail");
```

Switch Statement in C

Switch Statement

- An alternative to multiple if-else.
- Executes one block based on **matching value**.
- Works only with int and char.

```
switch(choice) {  
    case 1: printf("Add"); break;  
    case 2: printf("Subtract"); break;
```

```

    case 3: printf("Multiply"); break;
    default: printf("Invalid choice");
}

```

- **break** is used to stop execution after a case.
- If **break** is missing, execution **falls through** to the next case.

C - Loops

2. Loops in C

Loops allow repeating a block of code **multiple times**.

(a) for Loop

- Used when the **number of iterations is known**.

```

for (int i = 1; i <= 5; i++) {
    printf("%d ", i);
}

```

(b) while Loop

- Used when the **number of iterations is unknown**, executes until condition becomes false.

```

int i = 1;
while (i <= 5) {
    printf("%d ", i);
    i++;
}

```

(c) do-while Loop

- Similar to while loop, but **executes at least once** because condition is checked at the end.

```

int i = 1;
do {
    printf("%d ", i);
    i++;
} while (i <= 5);

```

Break Statement

break Statement

- Used inside loops/switch to **exit immediately**.

```

for(int i=1; i<=5; i++) {

```

```

        if(i==3) break;
        printf("%d ", i);
    }

```

Output: 1 2

continue Statement

- Skips current iteration and moves to the next.

```

for(int i=1; i<=5; i++) {
    if(i==3) continue;
    printf("%d ", i);
}

```

Output: 1 2 4 5

goto Statement

```

goto label;

/* some statements */

label:

    statement;

```

- Label: A user-defined identifier followed by a colon (:).
- The `goto` statement transfers control unconditionally to the labeled statement.

Arrays in C

- An **array** is a collection of elements of the **same data type** stored in **continuous memory**.
- Each element is accessed using an **index**, starting from 0.

Types of Arrays

1. **One-dimensional (1D)** → Like a list
2. **Two-dimensional (2D)** → Like a table/matrix
3. **Multi-dimensional** → Higher dimensions

1D Array Example:

```

int marks[5] = {90, 80, 70, 85, 95};
printf("%d", marks[2]); // Output: 70

```

2D Array Example (Matrix):

```

int mat[2][3] = {{1, 2, 3}, {4, 5, 6}};

```

```
printf("%d", mat[1][2]); // Output: 6
```

3D array Example:

A **3D array** is an array of 2D arrays.

You can think of it as a **cube** (rows × columns × depth).

```
#include <stdio.h>
int main() {
    int cube[2][2][2] = {
        { {1, 2}, {3, 4} },
        { {5, 6}, {7, 8} }
    };

    printf("Element at [1][0][1] = %d\n", cube[1][0][1]); //
Accessing

    return 0;
}
```

- cube [2][2][2] → 2 blocks, each block has 2 rows and 2 columns.
- Structure:

Block 0:

```
1 2
3 4
```

Block 1:

```
5 6
7 8
```

- cube[1][0][1] →
 - Block **1** (second block)
 - Row **0** (first row)
 - Column **1** (second column)→ Value = **6**

Output

```
Element at [1][0][1] = 6
```

Advantages of Arrays

- Stores multiple values with a single name.
- Easy to process with loops.
- Reduces code length.

Strings in C

- A **string** is a sequence of characters **terminated by '\0'** (null character).
- Strings are actually **character arrays**.

Example:

```
char name[10] = "Hello";  
printf("%s", name); // Output: Hello
```

String Input/Output:

```
char str[20];  
scanf("%s", str); // reads word until space  
printf("%s", str); // prints string
```

To read a full sentence with spaces, use `gets(str)` (deprecated, safer alternative is `fgets`).

String Handling Functions (<string.h>)

1. `strlen(str)` → returns length.
2. `strcpy(dest, src)` → copies string.
3. `strcat(str1, str2)` → concatenates.
4. `strcmp(str1, str2)` → compares.
5. `strupr(str) / strlwr(str)` → converts case (compiler dependent).

Summary

- **Control flow statements** control execution using decisions (`if`, `switch`) and loops (`for`, `while`, `do-while`).
- **Jump statements** (`break`, `continue`) alter normal flow inside loops.
- **Arrays** allow handling multiple values under a single variable name.
- **Strings** are special arrays of characters ending with '`\0`' and are supported by powerful library functions.

Control Flow Statements Type	Control Flow Statement	Description
Conditional Statements	if-else	Executes a block of code if a specified condition is true, and another block if the condition is false.
	switch-case	Evaluates a variable or expression and executes code based on matching cases.

Looping Statements	<code>for</code>	Executes a block of code a specified number of times, typically iterating over a range of values.
	<code>while</code>	Executes a block of code as long as a specified condition is true.
	<code>do-while</code>	Executes a block of code once and then repeats the execution as long as a specified condition is true.
Jump Statements	<code>break</code>	Terminates the loop or switch statement and transfers control to the statement immediately following the loop or switch.
	<code>continue</code>	Skips the current iteration of a loop and continues with the next iteration.
	<code>return</code>	Exits a function and returns a value to the caller.
	<code>goto</code>	Transfers control to a labeled statement within the same function. (Note: <code>goto</code> is generally discouraged due to its potential for creating unreadable and error-prone code.)

UNIT III – QUESTION BANK

2 MARKS QUESTIONS

1. What are decision-making statements in C?
 2. Write the syntax of an if-else statement.
 3. What is the purpose of a switch statement?
 4. Differentiate between break and continue statements.
 5. Define a loop.
 6. What is an array?
 7. What is the difference between an array and a string?
 8. Write the syntax of a for loop.
 9. Define while and do...while loops.
 10. What is meant by string handling in C?
-

5 MARKS QUESTIONS

1. Explain **different types of loops** in C with syntax and examples.
 2. Write a short note on the **switch statement** with a simple program.
 3. Write a program to **find the sum of all elements in an array**.
 4. Explain the **difference between break and continue** with examples.
 5. Write a C program to **count the number of vowels in a string**.
-

10 MARKS QUESTIONS

1. Explain in detail about **decision-making statements** in C (if, if-else, nested if, else-if ladder, and switch) with syntax and examples.
2. Discuss all **looping constructs** (for, while, do-while) in C with flowcharts, syntax, and examples.
3. Explain **arrays** in C — declaration, initialization, and accessing elements — with a sample program.
4. Explain **string handling functions** in C (strlen, strcpy, strcat, strcmp) with examples.
5. Write a program to **read marks of 5 students using arrays** and display the average mark.

Unit IV

Multidimensional Arrays in C

A multidimensional array is essentially an array of arrays (e.g., a 2-D array is an array whose each element is itself an array). It can store data in table-like form (rows & columns).

Declaration Syntax:

```
data_type name[row_size][column_size];
```

Example Program:

```
#include <stdio.h>
int main() {
    int matrix[2][3] = { {10, 20, 30}, {40, 50, 60} };
    printf("Matrix elements:\n");
    for(int i=0; i<2; i++){
        for(int j=0; j<3; j++){
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

Expected Output:

```
Matrix elements:
10 20 30
40 50 60
```

String functions in C

Strings in C are character arrays ending with a null character '\0'. C's standard library (<string.h>) provides functions for common operations: strlen() (length), strcpy() (copy), strcat() (concatenate), strcmp() (compare), etc.

Example Program:

```
#include <stdio.h>
#include <string.h>

int main() {
    char str1[20] = "Hello";
    char str2[20] = "World";
    char combined[40];

    strcpy(combined, str1);           // copy str1 into combined
    strcat(combined, str2);           // append str2
}
```

```

    printf("Combined string: %s\n", combined);
    printf("Length: %lu\n", strlen(combined));
    printf("Compare \"%s\" and \"%s\": %d\n", str1, str2,
    strcmp(str1, str2));
    return 0;
}

```

Expected Output:

Combined string: HelloWorld

Length: 10

Compare "Hello" and "World": (some negative value)

Practice problems Functions in C:Function Prototype-Parameter Passing Techniques in

C-Storage Classes

- A **function prototype** is a declaration of a function earlier in the program (or in header) that tells the compiler about the function's return type, name, and parameter types before its actual definition.
- **Parameter passing techniques:**
 - *Call by value* — a copy of the argument value is passed; original not modified.
 - *Call by reference* — the address (pointer) is passed; modifications affect the original variable.
- **Storage classes** define scope, lifetime, and visibility of variables: auto, register, static, extern.

Example Program (prototype + parameter passing + storage class):

```

#include <stdio.h>

// function prototype
int add(int a, int b);

void modifyByRef(int *p) {
    *p = *p + 100;
}

int main() {
    int sum;
    int x = 50;

    sum = add(20, 30);
    printf("Sum = %d\n", sum);

    modifyByRef(&x);
    printf("After modifyByRef, x = %d\n", x);

    static int s = 0;    // static storage class
    s++;
}

```

```

        printf("static s = %d\n", s);

    return 0;
}

int add(int a, int b) {
    return a + b;
}

```

Expected Output:

```

Sum = 50
After modifyByRef, x = 150
static s = 1

```

C-Recursion Concept

Recursion is the technique where a function calls itself until a base condition is reached. It is useful for problems like factorial, Fibonacci series, etc.

Example Program (factorial):

```

#include <stdio.h>

int factorial(int n) {
    if (n == 0)
        return 1;
    else
        return n * factorial(n - 1);
}

int main() {
    int num = 5;
    printf("Factorial of %d = %d\n", num, factorial(num));
    return 0;
}

```

Expected Output:

```

Factorial of 5 = 120

```

UNIT IV – QUESTION BANK

2 MARKS QUESTIONS

1. Define a multidimensional array with an example.
2. What is a string in C?
3. Mention any two string handling functions in C.
4. What is a function prototype?
5. Define recursion.
6. List any two advantages of using functions in C.
7. What are storage classes in C?
8. Differentiate between auto and static variables.
9. What is the use of the extern keyword?
10. Give the syntax of declaring a 2D array.

5 MARKS QUESTIONS

1. Explain **multidimensional arrays** with an example program.
2. Write a program to **concatenate two strings** using string functions.
3. Explain different **storage classes** in C with examples.
4. Write a short note on **parameter passing techniques** in C.
5. What is **recursion**? Write a recursive function to find the **factorial of a number**.

10 MARKS QUESTIONS

1. Discuss in detail about **multidimensional arrays** in C.
 - Definition
 - Declaration and initialization
 - Example with output
 - Advantages
2. Explain the **string functions in C** with syntax and suitable examples for each (strlen, strcpy, strcat, strcmp).
3. Describe the **different types of user-defined functions** in C.
 - Function with no arguments and no return value
 - Function with arguments but no return value
 - Function with no arguments but return value
 - Function with arguments and return value(Include syntax and examples)

4. Explain **storage classes** in C with their scope and lifetime. Provide an example program showing the use of static and extern.
5. Define **recursion** and explain its working with a suitable example program (factorial or Fibonacci series).

UNIT V

Pointers in C

A **pointer** is a special variable that stores the **memory address** of another variable. Pointers make it possible to access and manipulate data stored at any memory location.

Need for Pointers:

- To access array elements efficiently
- To perform dynamic memory allocation
- To pass large structures or arrays to functions
- To implement data structures such as linked lists and trees

Declaration Syntax

```
data_type *pointer_name;
```

Example Program

```
#include <stdio.h>
int main() {
    int a = 10;
    int *p;
    p = &a; // pointer stores address of a
    printf("Value of a = %d\n", a);
    printf("Address of a = %p\n", &a);
    printf("Value of *p = %d\n", *p); // prints value stored
    at address
    return 0;
}
```

Output

```
Value of a = 10
Address of a = 0x7ffee0... (varies)
Value of *p = 10
```

Pointer vs Array in C

Aspect	Pointer	Array
Declaration	<code>int *p;</code>	<code>int a[5];</code>
Memory Allocation	Dynamic (runtime)	Static (compile time)
Access	<code>*(p + i)</code>	<code>a[i]</code>
Relation	Can point to array	Array name acts like a pointer

Example

```
#include <stdio.h>
int main() {
```

```

    int a[3] = {10, 20, 30};
    int *p = a;
    for(int i = 0; i < 3; i++)
        printf("%d ", *(p + i));
    return 0;
}

```

Output

10 20 30

Structures

A **structure** is a **user-defined data type** that groups variables of different types under one name.

Syntax

```

struct structure_name {
    data_type member1;
    data_type member2;
};

```

Example

```

#include <stdio.h>
struct Student {
    int id;
    char name[20];
    float marks;
};
int main() {
    struct Student s1 = {101, "Priya", 88.5};
    printf("ID: %d\nName: %s\nMarks: %.2f\n", s1.id, s1.name,
s1.marks);
    return 0;
}

```

Output

ID: 101
Name: Priya
Marks: 88.50

Unions

A **union** is similar to a structure, but all members share the **same memory location**. Only **one member** can hold a value at a time.

Syntax

```

union union_name {
    data_type member1;
    data_type member2;
};

```

Example

```
#include <stdio.h>
union Data {
    int i;
    float f;
};
int main() {
    union Data d;
    d.i = 10;
    printf("Integer: %d\n", d.i);
    d.f = 5.5;        // overwrites i
    printf("Float: %.1f\n", d.f);
    return 0;
}
```

Output

```
Integer: 10
Float: 5.5
```

Enumeration (or enum) in C

enum defines a set of named integer constants, improving readability and clarity.

Example

```
#include <stdio.h>
enum Week {Sun, Mon, Tue, Wed, Thu, Fri, Sat};
int main() {
    enum Week day;
    day = Wed;
    printf("Day number = %d\n", day);
    return 0;
}
```

Output

```
Day number = 3
```

C application programs

Sorting

```
#include <stdio.h>
int main() {
    int a[5] = {5, 2, 8, 1, 3}, i, j, temp;
    for(i = 0; i < 5; i++)
        for(j = i + 1; j < 5; j++)
```

```

        if(*(a + i) > *(a + j)) {
            temp = *(a + i);
            *(a + i) = *(a + j);
            *(a + j) = temp;
        }
    printf("Sorted: ");
    for(i = 0; i < 5; i++)
        printf("%d ", *(a + i));
    return 0;
}

```

Output

Sorted: 1 2 3 5 8

Matrix manipulations

```

#include <stdio.h>
int main() {
    int a[2][2] = {{1,2},{3,4}}, b[2][2] = {{5,6},{7,8}},
    sum[2][2];
    for(int i=0;i<2;i++)
        for(int j=0;j<2;j++)
            sum[i][j] = a[i][j] + b[i][j];
    printf("Result:\n");
    for(int i=0;i<2;i++){
        for(int j=0;j<2;j++)
            printf("%d ", sum[i][j]);
        printf("\n");
    }
    return 0;
}

```

Output

Result:
6 8
10 12

Student's Mark list preparation

```

#include <stdio.h>
struct Student {
    char name[20];
    int m1, m2, m3;
    float total, avg;
};
int main() {
    struct Student s = {"Kaviya", 80, 75, 90};
    s.total = s.m1 + s.m2 + s.m3;
    s.avg = s.total / 3;
}

```

```
    printf("Name: %s\nTotal: %.2f\nAverage: %.2f\n", s.name,  
s.total, s.avg);  
    return 0;  
}
```

Output

```
Name: Kaviya  
Total: 245.00  
Average: 81.67
```

UNIT V – QUESTION BANK

2 MARKS QUESTIONS

1. What is a pointer in C?
 2. Write the syntax for pointer declaration.
 3. What is the & (address-of) operator used for?
 4. Define a structure in C.
 5. What is the difference between a structure and a union?
 6. Define an enumeration (enum) with an example.
 7. What is the relationship between a pointer and an array?
 8. Mention any two applications of pointers.
 9. Give an example of accessing structure members using the dot (.) operator.
 10. What is the use of the -> (arrow) operator in structures?
-

5 MARKS QUESTIONS

1. Explain **pointer declaration and initialization** with an example program.
 2. Write a program to **swap two numbers using pointers**.
 3. Explain **structures** with syntax and example.
 4. Differentiate between **structure and union** with suitable examples.
 5. Write a short note on **enumeration (enum)** in C with a sample program.
-

10 MARKS QUESTIONS

1. Explain **pointers** in C in detail — definition, declaration, initialization, and pointer arithmetic — with examples.
2. Explain **structures in C** — declaration, initialization, accessing members, and structure within structure — with example programs.
3. Explain **unions in C** — syntax, example, and difference between structure and union.
4. Write a program using **pointers and arrays** to sort elements in ascending order.
5. Write a C program to **store and display student details (name, roll number, and marks)** using **structures and arrays**.